

Load Balancer para Modelos de IA

Artur Fernandes Braga de Menezes¹, Artur Rizzi Martinho¹,
Felipe Lacerda Tertuliano¹, Gabriel Alves da Silva Diógenes¹,
Rafael Mortimer Colares¹

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-901 - Belo Horizonte - MG - Brazil

Abstract. *Ullamco nostrud aliquip proident ex ullamco ullamco nostrud adipiscing. Non commodo incididunt veniam pariatur veniam sit. Duis consequat esse enim deserunt dolor exercitation. Quis consequat velit deserunt sint. Exercitation deserunt labore veniam quis ad enim quis commodo culpa incididunt aute veniam voluptate. Dolor aliqua aliqua non aliqua. In ex aute eiusmod voluptate exercitation proident eu minim magna aliqua fugiat mollit et veniam.*

Resumo. *Ullamco nostrud aliquip proident ex ullamco ullamco nostrud adipiscing. Non commodo incididunt veniam pariatur veniam sit. Duis consequat esse enim deserunt dolor exercitation. Quis consequat velit deserunt sint. Exercitation deserunt labore veniam quis ad enim quis commodo culpa incididunt aute veniam voluptate. Dolor aliqua aliqua non aliqua. In ex aute eiusmod voluptate exercitation proident eu minim magna aliqua fugiat mollit et veniam.*

1. Introdução

A evolução dos modelos de Inteligência Artificial (IA), especialmente dos Grandes Modelos de Linguagem (Large Language Models — LLMs), tem ampliado significativamente a capacidade dos sistemas computacionais de realizar tarefas como geração e compreensão de texto, programação e processamento de informações. Entretanto, o aumento da capacidade desses modelos também está associado a uma maior demanda por recursos computacionais durante sua utilização. Além disso, diferentes aplicações podem apresentar requisitos distintos de processamento, latência e utilização de recursos, tornando o gerenciamento eficiente das requisições um desafio relevante para sistemas que disponibilizam modelos de IA em larga escala. Estudos sobre sistemas de serving de LLMs destacam justamente a heterogeneidade das requisições e a necessidade de mecanismos eficientes para distribuir e agendar sua execução [Kossmann et al. 2024, Sun et al. 2024].

Nesse contexto, utilizar um único modelo de grande porte para atender diferentes tipos de requisições pode resultar em uma utilização pouco eficiente dos recursos disponíveis, principalmente quando determinadas tarefas poderiam ser executadas por modelos menores ou especializados. O problema também se estende à distribuição das requisições entre diferentes instâncias, uma vez que variações na demanda podem provocar sobrecarga em determinados recursos enquanto outros permanecem subutilizados. Trabalhos recentes demonstram que estratégias de balanceamento e escalonamento podem melhorar a utilização da infraestrutura disponível. O SkyWalker, por exemplo, propõe um mecanismo de balanceamento de carga entre diferentes regiões para inferência de LLMs, buscando utilizar os recursos de forma mais eficiente e reduzir custos de serving [Xia et al. 2025]. De forma semelhante, o Llumnix explora o reagendamento dinâmico

de requisições entre múltiplas instâncias de modelos para lidar com diferentes requisitos de recursos e latência [Sun et al. 2024].

Diante desse cenário, este trabalho propõe o estudo e desenvolvimento de um Load Balancer voltado para modelos de IA, capaz de distribuir as requisições entre diferentes modelos de acordo com suas características e com a disponibilidade dos recursos. A utilização de múltiplos modelos pode permitir que diferentes tipos de solicitações sejam direcionados para alternativas mais adequadas, evitando o uso desnecessário de modelos de maior porte quando modelos mais simples forem suficientes para determinada tarefa. Dessa forma, busca-se investigar os possíveis impactos dessa abordagem sobre aspectos como utilização dos recursos computacionais, tempo de resposta e custo de processamento. A literatura demonstra que mecanismos de balanceamento e escalonamento aplicados ao serving de LLMs podem proporcionar ganhos de desempenho e redução de custos, reforçando a relevância de investigar estratégias semelhantes em arquiteturas compostas por múltiplos modelos [Kossmann et al. 2024, Xia et al. 2025, Sun et al. 2024].

2. Revisão bibliográfica

Sistemas de inferência de LLMs não tratam todas as requisições da mesma forma. Há prompts curtos e longos, respostas de poucos tokens e de milhares, picos de demanda e períodos ociosos. Por isso, o load balancer deixou de ser um detalhe de infraestrutura e passou a definir latência, custo e uso de GPU. Os cinco trabalhos a seguir discutem justamente esse componente: a quem enviar cada requisição. Nenhum deles, porém, escolhe entre modelos de portes diferentes, que é o recorte deste projeto.

Kossmann et al. descrevem a arquitetura mais comum desses sistemas em dois níveis [Kossmann et al. 2024]. Na entrada, o load balancer escolhe o servidor que guarda uma réplica do modelo. Dentro do servidor, um scheduler decide se a requisição espera na fila, entra no lote de execução ou é interrompida. Na prática, muitos serviços ainda usam round-robin, escolha aleatória ou *power of two choices*. Os autores propõem o SAL (*Server-Aware Load Balancer*), que estima a carga de cada máquina a partir dos tokens ainda na fila e da memória livre, e envia o pedido para a menos carregada. O SAL é simples de colocar no lugar da política atual e reduz o tempo de geração em relação a balanceadores genéricos. O artigo liga redes, ou seja, qual servidor recebe o pedido, a sistemas operacionais, isto é, como a fila e a preempção funcionam na GPU.

Jain et al. mostram que balancear entre várias instâncias do mesmo modelo pesa mais na latência do que só ajustar o scheduler de uma GPU [Jain et al. 2025]. Round-robin e políticas parecidas tratam a inferência como um job único e ignoram as duas fases do processamento: o *prefill*, mais pesado em computação, e o *decode*, mais pesado em memória. Misturar esses perfis na mesma instância atrasa todo mundo. O load balancer deles prevê o tamanho da resposta e o efeito de misturar cargas. Em quatro instâncias, a latência média cai 11,43% em 2.000 requisições. A limitação é clara e importa para este trabalho: todas as instâncias servem o mesmo modelo. Não há escolha entre um modelo pequeno e um grande.

O Llumnix parte de outra observação: o tempo de serviço e o uso de memória de uma requisição de LLM não se conhecem de antemão [Sun et al. 2024]. O KV cache cresce a cada token, as prioridades mudam e um único despacho no início não dá conta. O sistema reagenda pedidos em execução, migrando a requisição e o estado na GPU de

uma instância para outra, como a troca de contexto entre núcleos no sistema operacional. Isso reduz interferência, fragmentação de memória e violação de SLO. Num cluster de 16 GPUs, a latência P99 do primeiro token cai até $15\times$ frente a um scheduler clássico. O ganho de custo chega a 36% para a mesma latência de cauda. Aqui o load balancing aparece como problema de sistemas operacionais: fila, isolamento, preempção e migração de estado.

O SkyWalker leva o mesmo problema para a rede geográfica [Xia et al. 2025]. Provedores reservam GPU em cada região para o pico local e desperdiçam capacidade fora do horário de pico. Um balanceador único no centro do mapa barateia o serviço, mas vira gargalo e ponto único de falha. A proposta coloca um load balancer por região e só desvia tráfego quando uma região satura. Round-robin e *least-load* falham porque o tempo de uma inferência varia de segundos a dezenas de segundos, e porque pedidos com o mesmo prefixo deveriam cair na GPU que já tem o KV cache. Em cargas reais, o SkyWalker aumenta a vazão em 1,12 a 2,06 $\times$ e reduz a latência em 1,74 a 6,30 $\times$ diante de load balancers existentes, com cerca de 25% menos custo. Para este projeto, o recorte multi-região fica fora do protótipo. O que permanece é a lição de rede: o despacho precisa olhar latência, vazão e estado dos backends, não só um rodízio cego.

O DualMap trata o conflito entre duas políticas que os artigos anteriores encostam, mas não unem de fato [Yuan et al. 2026]. Mandar pedidos com o mesmo prefixo para a mesma máquina melhora o cache e o tempo até o primeiro token, mas cria pontos quentes. Espalhar pela menor carga equilibra o cluster e quebra o cache. A solução mapeia cada requisição a duas instâncias candidatas, a ideia clássica de *power of two choices*, e escolhe a melhor no momento. Se o SLO de TTFT ameaça estourar, o sistema deixa o cache em segundo plano e passa a olhar carga. Se uma máquina esquentar, migra pedidos para a instância reserva. Sobre o vLLM, a capacidade efetiva de requisições sobe até 2,25 $\times$ com o mesmo SLO. O artigo une redes, no despacho e na elasticidade, e arquitetura, no KV cache e na ocupação de GPU.

Os cinco trabalhos concordam em três pontos. Load balancing de LLM não se comporta como o de um site comum: o serviço é longo, tem duas fases e o tamanho da resposta não é conhecido. O balanceador e o scheduler interno são camadas diferentes; otimizar só uma não basta [Kossmann et al. 2024, Jain et al. 2025]. Memória, KV cache, fragmentação e localidade de prefixo entram na decisão [Sun et al. 2024, Xia et al. 2025, Yuan et al. 2026].

A lacuna que este trabalho explora está no que todos eles deixam de fora. Os cinco balanceiam réplicas do mesmo modelo, ou do mesmo serviço, e perguntam apenas *qual instância* deve atender o pedido. Nenhum classifica a complexidade da consulta para escolher entre um modelo leve e um modelo maior, nem junta essa escolha com o estado das filas. O load balancer proposto aqui continua sendo um despachante de camada de aplicação, mas o conjunto de destinos passa a depender do tipo de prompt, e a instância escolhida, da carga observada.

3. Metodologia de Desenvolvimento e Avaliação

A metodologia adotada neste trabalho organiza-se em torno de ciclos semanais de desenvolvimento e avaliação, conforme apresentado na Tabela 1. Se espera ter até a sexta semana um código completo capaz de atender nossos requisitos e ser utilizado para as

medições, a partir disso o foco do trabalho se vira para coleta de resultados, melhorias, redação do artigo e apresentações.

Tabela 1. Cronograma de atividades do projeto.

Semana	Atividades principais
1	Configuração do ambiente (Ollama, modelos leves), levantamento bibliográfico inicial.
2	Desenvolvimento do classificador de prompts, implementação da versão inicial do LoadBalancer.
3 (Sprint 2)	Detalhamento da implementação, coleta de resultados preliminares, preparação e apresentação oral do Sprint 2.
4	Refinamento do classificador, implementação de métricas de monitoramento, escrita da metodologia.
5	Expansão do conjunto de testes, execução de experimentos comparativos, análise estatística dos resultados.
6	Otimização do LoadBalancer, testes automatizados de roteamento, escrita da seção de resultados.
7	Comparação com trabalhos relacionados, redação da seção comparativa, preparação de material visual para o pitch.
8	Redação das conclusões, revisão e refinamento do artigo, roteiro do pitch comercial.
9 (Sprint 3)	Gravação e edição do pitch comercial, preparação e apresentação oral final.
10	Submissão final do artigo, documentação do projeto e entrega dos materiais de apresentação.

Quanto ao método de avaliação e validação foi escolhida a medição em ambiente controlado complementada pela análise dos resultados, isso porque o desempenho de um LoadBalancer para modelos de IA depende de fatores dinâmicos do ambiente que não podem ser capturados por simulações. A validação será conduzida por meio de experimentos comparativos entre dois cenários: (i) *baseline*, no qual todas as requisições são direcionadas ao modelo de maior porte; e (ii) *proposta*, na qual o LoadBalancer classifica o prompt e o encaminha ao modelo mais adequado. Serão coletadas métricas de latência média, P95 e P99, além de uso de RAM e GPU por máquina, a análise estatística dos dados incluirá identificação de outliers e geração de gráficos comparativos.

O ambiente experimental será composto por máquinas executando o Ollama com modelos leves (Phi-3.5-mini, Qwen2.5-Coder-0.5B e AILO-152M-v2), e o LoadBalancer será implementado em Python com FastAPI, utilizando `httpx` para comunicação com os servidores de inferência. Ajustes podem envolver a inclusão de novas métricas, a alteração do conjunto de modelos avaliados ou a adoção complementar de técnicas de simulação, caso se mostrem necessárias para validar aspectos específicos do sistema.

4. Resultados

Consectetur quis cillum eu minim Lorem minim anim dolor et. Ut incididunt est enim Lorem commodo occaecat sunt nulla adipisicing fugiat. Quis amet nisi do deserunt laborum. Labore velit deserunt excepteur et velit adipisicing consectetur ipsum. Enim dolore

eiusmod proident exercitation ullamco commodo veniam anim ad ut ullamco Lorem Lorem.

5. Conclusão

Nulla velit laboris ut in in ullamco minim id cillum magna quis ullamco in. Veniam proident proident reprehenderit nostrud aliqua incididunt adipisicing ullamco deserunt irure proident ea officia. Tempor elit minim voluptate adipisicing laborum.

Referências

- Jain, K., Parayil, A., Mallick, A., Choukse, E., Qin, X., Zhang, J., Íñigo Goiri, Wang, R., Bansal, C., Rühle, V., Kulkarni, A., Kofsky, S., and Rajmohan, S. (2025). Performance aware llm load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 19–30. Association for Computing Machinery.
- Kossmann, F., Fontaine, B., Khudia, D., Cafarella, M., and Madden, S. (2024). Is the gpu half-empty or half-full? practical scheduling techniques for llms. *arXiv preprint arXiv:2410.17840*.
- Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. (2024). Llum-nix: Dynamic scheduling for large language model serving. *arXiv preprint arXiv:2406.03243*.
- Xia, T., Mao, Z., Kerney, J., Jackson, E. J., Li, Z., Xing, J., Shenker, S., and Stoica, I. (2025). Skywalker: A locality-aware cross-region load balancer for llm inference. *arXiv preprint arXiv:2505.24095*.
- Yuan, Y., Zuo, P., Wang, B., Chen, Z., Tan, Z., and Yu, Z. (2026). Dualmap: Enabling both cache affinity and load balancing for distributed llm serving. *arXiv preprint arXiv:2602.06502*.